
Metal Orbit-Recurrent Packed Hypercode (MORPH-32)

Elwan Mayencourt
Technical report, September 2026

Codex (GPT-6 Astra High)
Technical report, September 2026

Abstract

Weight-only quantization makes large language models easier to deploy, but further compression must preserve useful predictions and admit an efficient decoder. **MORPH-32** introduces a post-training weight representation and inference system for Apple M5. MORPH stands for **Metal Orbit-Recurrent Packed Hypercode**. Metal identifies the execution backend, orbit refers to the compact layout’s rotation-based bit coupling, and 32 is the number of weights sharing a code and scale. MORPH32-c stores two sign planes and half of a third; rotation and XOR reconstruct the missing bits. MORPH32-3s stores all three planes directly as a simpler option. Custom kernels consume both layouts directly. On Qwen3.8-27B, MORPH reduces complete checkpoint size by 14.99%–18.31% relative to MLX 4-bit and saves up to 2.95 GB of loaded model arrays. The compact profile scores 82.46% on a 1,140-question MMLU subset; the simpler full profile scores 83.60%, versus 84.04% for MLX 4-bit. Both profiles exceed MLX 3-bit on MMLU, HumanEval and text/code prediction metrics, at larger size and lower throughput. Six primary configurations, exact metadata controls and a second-model study establish where the savings come from and where quality is lost. MORPH thus provides a working compressed representation whose storage, reconstruction and execution can be studied together, with measured compression benefits rather than nominal bit counts alone.

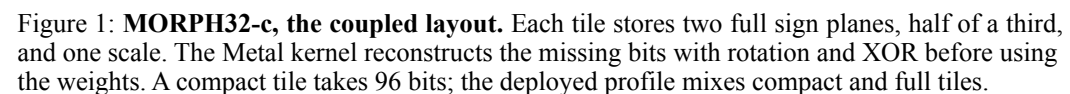
1 Introduction

Low-bit weight quantization has become a practical route to running large language models on local hardware. Reducing weight precision lowers the model’s storage footprint and the amount of weight data an inference kernel must access. The remaining challenge is to choose a representation that offers a useful combination of compression, prediction quality and execution cost. A nominal bit rate describes only part of that choice: scales, retained tensors, model assets and reconstruction work also matter.

Recent methods improve different parts of this problem. GPTQ uses second-order information to choose quantized weights; AWQ protects influential channels through activation-aware scaling; AQLM and QuIP# design richer representations for groups of weights. Their systems results also make an important distinction: a compact representation needs an implementation that can exploit it. MLX already supplies packed quantized matrix multiplication on Apple Silicon. MORPH investigates a different stored layout and reconstruction rule within that setting, rather than claiming direct quantized execution itself as new.

The central idea is to give a 32-weight tile a small shared state that a Metal kernel can interpret directly. MORPH32-c stores two sign planes, half of a third and one scale; rotation and XOR rebuild the missing half as the weights are used. MORPH32-3s is the simpler layout that stores all three sign planes. Activation-weighted fitting selects the stored state, and a standalone container combines the resulting MLP weights with retained tensors and losslessly packed metadata.

The system makes three contributions: full and compact tile layouts with explicit reconstruction equations and exact storage costs; an encoder, standalone packaging and direct Metal execution for generation and prefill; and a complete evaluation with common inputs, capability tests and controlled ablations that separates representation savings from lossless metadata savings. The second-model and kernel experiments also identify limits that a broader compression claim would need to overcome.



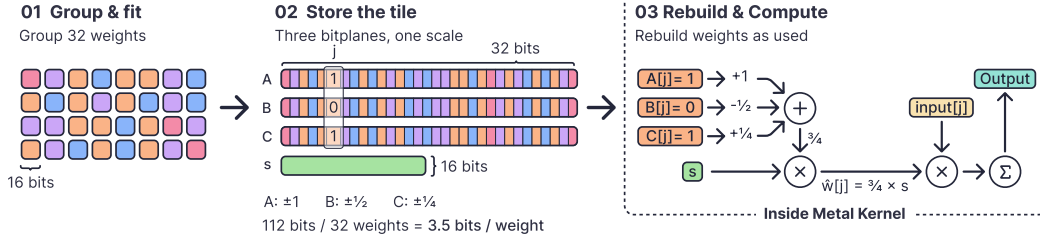


Figure 2: **MORPH32-3s, the simpler full layout.** All three sign planes are independent and stored directly. Reconstruction uses the same weight equation as MORPH32-c, without rotation or XOR.

2 Background and related work

Quantization algorithms GPTQ (Frantar et al., 2023) uses approximate second-order information and error compensation to fit low-bit weights. AWQ (Lin et al., 2024) uses activation statistics to identify influential channels and searches for protective scaling. SmoothQuant (Xiao et al., 2023) instead targets weight-and-activation quantization through an equivalent redistribution of channel scales. MORPH is weight-only: it uses activation energy to weight fitting errors, but does not quantize activations or implement these algorithms.

Representations for groups of weights AQLM (Egiazarian et al., 2024) approximates weight groups with sums of learned codebook entries and tunes the codes and codebooks against layer outputs. QuIP# (Tseng et al., 2024) combines randomized Hadamard processing, structured lattice codebooks and fine-tuning. Binary combinations also precede these LLM methods, including ABC-Net (Lin et al., 2017). MORPH’s full tile is a symmetric scalar grid expressed as sign planes; its compact tile imposes a cross-position constraint through bit reconstruction. It uses neither learned additive codebooks nor a Hadamard transform.

Containers and execution GGUF specifies how tensors and model metadata are packaged for GGML-based inference; it is not a single quantization algorithm. MLX specifies groupwise quantization and matrix multiplication over packed codes, scales and biases. These address different levels of the deployment stack. MORPH likewise specifies both a weight representation and its container/runtime integration. Its empirical baselines are the measured MLX 4-bit and MLX 3-bit checkpoints. The papers and GGUF specification provide context; their published benchmark values are not mixed into the MORPH results.

3 MORPH-32: representation and fitting

The format is designed around a fixed tile size, shared scale and decoder with no learned lookup table. “Packed Hypercode” names the tile’s joint stored state. “Orbit” describes the rotation-based coupling in the compact tile. The simpler full tile reads all three planes directly.

3.1 A shared three-plane decoder

For an $N \times K$ projection, split each output row into contiguous tiles of 32 input weights. Bit j refers to position j within the tile, least-significant-bit first. Let $u_{j(z)} = 2b_{j(z)} - 1$, where $b_{j(z)}$ extracts a bit from word z . The reconstructed weight is

$$\hat{w}_j = s \left(u_{j(A)} + \frac{1}{2} u_{j(B)} + \frac{1}{4} u_{j(C)} \right). \quad (1)$$

The three sign choices produce a symmetric eight-level grid, scaled independently for each tile. For example, bits (1, 0, 1) give $\hat{w}_j = 0.75s$. Both profiles use this same weight equation, without a learned codebook or per-tile offset. They differ in how the third sign plane is stored.

3.2 MORPH32-c: a coupled third plane

A compact tile retains words A , B and the low 16 bits L of C . Its upper half is determined by

$$H = (L \oplus A \oplus \text{rotr}(B, 5)) \text{ and } 65535, \quad C = L + 2^{16}H. \quad (2)$$

All operations are unsigned and rotation wraps at 32 bits. This fixed relation saves 16 stored bits per tile and couples the choices available at different weight positions. The reconstructed C then enters (1). The tile stores 96 bits, including an FP16 scale, for 3.0 bits/weight.

On Qwen3.8-27B, MORPH32-c uses compact tiles in the 96 gate/up projections of layers 8–55, with zero-based indexing. The remaining gate/up projections and every down projection use full tiles. This fixed mixed layout averages 3.25 bits/MLP weight; it is not an adaptive choice made during inference.

3.3 MORPH32-3s: the simpler full tile

MORPH32-3s stores all three 32-bit words and the FP16 scale. Its 112-bit tile represents 32 weights at 3.5 bits/weight. No bits of C are generated from other positions. Both public profiles convert all 192 language MLP projections.

Table 1: **Representation cost.** Rates include the tile scale. The mixed compact profile averages 3.25 bits per MLP weight; complete-checkpoint size additionally includes retained tensors and model assets.

Tile layout	Stored state	Bits/tile	Bits/weight
Compact	$2 \times \text{uint32} + \text{uint16} + \text{FP16}$	96	3.00
Full	$3 \times \text{uint32} + \text{FP16}$	112	3.50

3.4 Fitting the representation

The encoder targets BF16 weights using calibration activations from MLX 4-bit. For each tile it minimizes

$$L(s, p) = \sum_{j=0}^{31} h_j (w_j - sv_{j(p)})^2, \quad (3)$$

where h_j is normalized input-channel mean squared activation and p contains the independent stored bits. Gate and up projections share their input statistics. This diagonal objective emphasizes influential channels while keeping tile fitting inexpensive.

Initialization tries clipping factors 0.70, 0.85 and 1.00, followed by two coordinate-search sweeps and FP16 scale refitting. Public profiles add two refinement sweeps and three deterministic randomized restarts, seed 2026. Candidates are compared using the scale as actually stored in FP16. The base-fit control omits this extra search, allowing the benefit of additional encoder work to be measured separately from the format itself.

4 Direct inference and checkpoint packaging

The representation is paired with a standalone loader and two execution paths: a dot-product kernel for generation and a matrix kernel for prefill. MORPH replaces the MLP projections and retains the other tensors from the MLX 4-bit checkpoint. Supported BF16 scale/bias metadata is packed losslessly using bit-pattern offsets, correlated magnitude prediction and sparse residuals. Packing is accepted only when smaller and exactly reconstructible. The `.morph` safetensors container includes projection descriptors, SHA-256 payload hashes, model configuration and tokenizer assets. Inference therefore needs only the converted file.

For generation, a SIMD lane reconstructs a 32-weight tile in registers, multiplies by the corresponding activations and participates in a 32-lane reduction. For prefill, kernels reconstruct 64×64 tiles into padded threadgroup memory and use M5 NAX matrix operations. Supported retained one-token projections fuse metadata decoding with the product; other shapes recover scale/bias arrays and use native MLX quantized multiplication. Small temporary tiles and metadata arrays are distinct from a complete dense MLP allocation.

5 Evaluation protocol

The evaluation runs Qwen3.8-27B on an Apple M5 Pro with 20 GPU cores and 48 GiB unified memory, using MLX 0.32.2 and MLX-LM 0.31.3. The comparisons comprise MLX 4-bit, MORPH32-c, MORPH32-3s, an MLX 3-bit checkpoint, and two 3s controls: base fitting and metadata packing disabled. Qwen3-0.6B provides a second-model check of size, prediction metrics and throughput.

The MLX 3-bit model uses MLX quantization with groups of 64. It shares the primary model’s architecture, tokenizer and chat template, but quantizes a broader language tensor scope and retains BF16 vision weights. Actual complete checkpoint sizes establish the storage comparison; nominal bit counts alone do not imply matched storage. All inference tests use the text path; complete files also contain vision tensors.

Table 2: **A common evaluation protocol.** Calibration uses a separate 128-token validation segment. Revisions, offsets, decoding details and provenance are specified in Appendix A.

Measurement	Protocol
Text / Python prediction	4,096 tokens per domain; WikiText-2 test and CodeSearchNet Python test
MMLU	1,140 fixed questions; 20 per subject, 57 subjects; five-shot choice likelihoods
HumanEval	164 tasks; one greedy completion each; 512-token cap; official tests
Positional retrieval	10 codes $\times$ 5 positions; approximately 17.6k tokens per prompt
Generation / prefill	10 runs; batch 1; 4,096 prompt + 64 generated tokens

Perplexity measures prediction quality on the evaluated text; lower is better. KL divergence and top-1 token agreement measure changes relative to MLX 4-bit, which is the reference rather than a full-precision oracle. MMLU and HumanEval measure task success. Capability intervals are descriptive Wilson 95% intervals. Timing uses medians and interquartile ranges (IQR) from sequential sessions; these ranges do not capture cross-session thermal or clock drift. All sizes use decimal GB.

6 Experimental results

6.1 Compression and runtime cost

MORPH32-c saves 2.95 GB of complete storage and 2.95 GB of loaded arrays versus MLX 4-bit. The simpler MORPH32-3s saves 2.41 and 2.41 GB, respectively. The complete-file reductions are 18.31% for MORPH32-c and 14.99% for MORPH32-3s.

Table 3: **Measured comparison of the four deployment choices.** RAM is resident model arrays; HE is HumanEval; generation is median tokens/s over ten runs. File size includes all tensors and assets. MLX 3-bit quantizes a broader tensor scope.

Model	File GB	RAM GB	Text PPL ↓	Code PPL ↓	MMLU ↑	HE ↑	Gen. ↑
MLX 4-bit	16.08	15.13	4.756	2.838	84.04%	76.83%	15.92
MORPH32-c	13.14	12.19	4.990	2.971	82.46%	70.12%	13.35
MORPH32-3s	13.67	12.72	4.920	2.915	83.60%	70.12%	14.64
MLX 3-bit	12.72	11.77	5.217	3.037	78.60%	64.02%	22.26

The reduction has two independently measurable sources. Replacing the MLP representation saves 2.14 GB for MORPH32-3s and 2.67 GB for MORPH32-c. Lossless metadata packing saves another 272.04 MB of payload. Container overhead explains the small difference between their sum and the complete-file saving. This accounting connects the tile bit rates to the deployed model size.

Direct execution preserves the storage benefit during loading, but decoding is not free. Generation reaches 14.64 and 13.35 tokens/s, versus 15.92 for MLX 4-bit; prefill reaches 366.16 and 348.88, versus 417.71. The MLX 3-bit model reaches 22.26 tokens/s. These are measured operating points, not isolated kernel-efficiency estimates.

6.2 Prediction quality across text and Python

Both prediction domains show the same quality ordering: MLX 4-bit, MORPH32-3s, MORPH32-c, then MLX 3-bit. Text perplexity is 4.756, 4.920, 4.990 and 5.217, respectively. Code perplexity follows the same pattern. Distribution-level results in Table 4 show how far each model moves from the MLX 4-bit reference.

Table 4: **Prediction and distribution fidelity, including both 3s controls.** PPL is perplexity; KL is mean divergence in nats/token from MLX 4-bit. Each domain contains 4,096 scored tokens. MLX 4-bit KL is zero by definition. NLL changes and token agreement appear in Appendix B.

Configuration	Text PPL ↓	Code PPL ↓	Text KL ↓	Code KL ↓
MLX 4-bit	4.756	2.838	0.0000	0.0000
MORPH32-c	4.990	2.971	0.1037	0.0732
MORPH32-3s	4.920	2.915	0.0787	0.0556
MLX 3-bit	5.217	3.037	0.1836	0.1233
3s, base fit	4.920	2.917	0.0791	0.0556
3s, metadata off	4.920	2.915	0.0787	0.0556

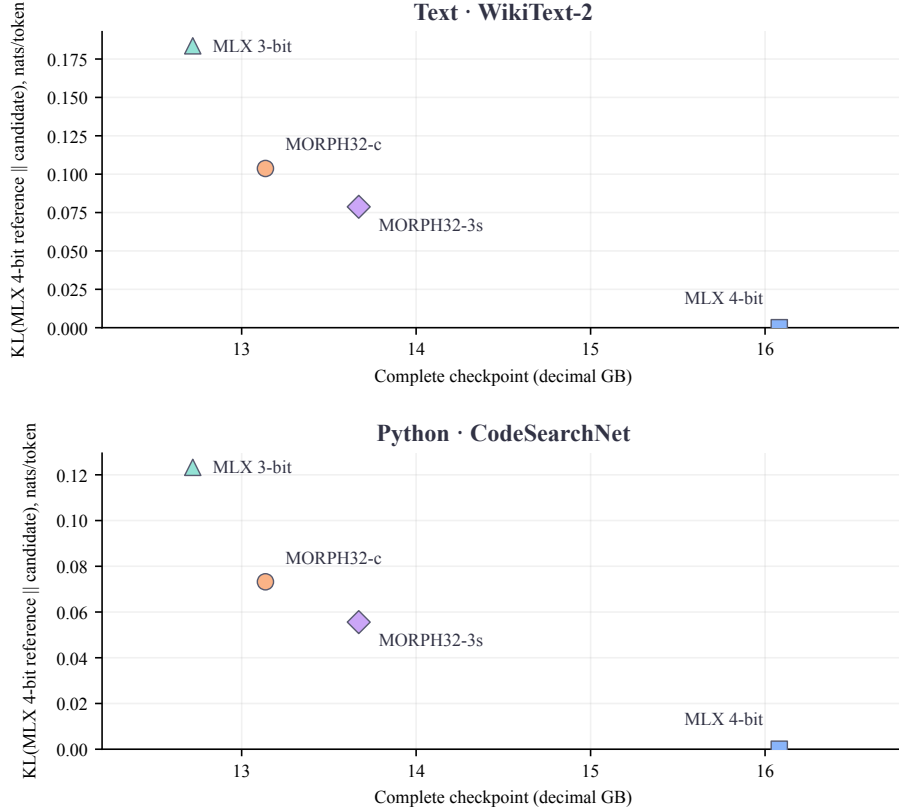


Figure 3: **Model size versus prediction divergence.** Four measured models are shown in both text and Python domains. Each point uses complete model file size and mean KL divergence from MLX 4-bit on the same 4,096-token input. Lower KL means closer predictions; the points do not define a continuous frontier.

6.3 Capability retention on knowledge and code

MORPH32-3s answers 953/1140 (83.60%) MMLU questions correctly, versus 958/1140 (84.04%) for MLX 4-bit. Its 0.44-percentage-point gap corresponds to 5 questions in this fixed subset. MORPH32-c answers 940/1140 (82.46%), while MLX 3-bit answers 896/1140 (78.60%). Both MORPH profiles pass 115 of 164 HumanEval tasks, compared with 126 for MLX 4-bit and 105 for MLX 3-bit.

MORPH’s gain over the MLX 3-bit checkpoint is present in both tasks, alongside its larger size. The result supports the format as a useful quality–storage choice, without claiming equal-storage superiority or equivalence to MLX 4-bit. All six configurations pass 50/50 retrieval prompts and 10/10 complete code trials across five insertion positions. The trial-level interval is 72.25–100%; this synthetic ceiling result provides a basic retrieval check rather than a ranking of long-context ability.

Similar aggregate accuracy does not imply identical behavior. On MMLU, standard 3s loses 39 answers that MLX 4-bit gets right and gains 34 that MLX 4-bit gets wrong. Its net loss is only five, although 92 answer choices change in total. Compact loses 59 and gains 41; MLX 3-bit loses 108 and gains 46. This paired analysis makes the limits of “retained accuracy” visible without introducing an additional performance metric.

6.4 What the ablations establish

The controls use the same current 3s layout, calibration and retained tensor policy. Table 5 includes every primary configuration so that product results and ablations can be read together.

Table 5: **Complete primary comparison.** HE counts passes out of 164; MMLU uses 1,140 questions. Last two rows are controls for the 3s profile. Timing is sequential and should not be interpreted as an isolated effect of fitting or metadata packing.

Configuration	File GB	Text PPL	MMLU %	HE passes	Gen. tok/s
MLX 4-bit	16.08	4.756	84.04	126	15.92
MORPH32-c	13.14	4.990	82.46	115	13.35
MORPH32-3s	13.67	4.920	83.60	115	14.64
MLX 3-bit	12.72	5.217	78.60	105	22.26
3s, base fit	13.67	4.920	83.60	112	15.24
3s, metadata off	13.94	4.920	83.60	115	16.58

Lossless metadata packing Disabling packing adds 271.84 MB to the complete checkpoint. Encoder accounting reports 272.04 MB of metadata payload savings; the small difference is container overhead. All 192 MLP descriptors and 384 recorded word/scale hashes match. MMLU choice probabilities, HumanEval token sequences and retrieval outputs also match exactly. This is a directly verified saving independent of lossy MLP quantization.

Additional fitting Strong and base fitting obtain the same MMLU count; strong fitting passes three more HumanEval tasks. Prediction differences are small and point in different directions for text and code. More search is therefore an encoder option with mixed measured benefit, not the explanation for all of MORPH’s gains.

Second-model validation On Qwen3-0.6B, standard 3s reduces the file from 0.35 to 0.31 GB, but text perplexity rises from 14.520 to 21.258. Generation is 204.92 versus 219.65 tokens/s for MLX 4-bit. Base fitting is slightly better in perplexity. The storage behavior transfers; the primary model’s quality retention does not yet generalize to this smaller model.

6.5 Runtime optimization: what remains to improve

The direct decoder performs bit extraction, reconstruction and scaling in addition to moving weights. A focused experiment split its serial accumulation into two chains. The later run reaches 15.99 tokens/s, compared with 14.64, while text perplexity, stored bytes and loaded arrays are unchanged. However, prefill also accelerates despite an unchanged prefill kernel, and synchronized projection-family timings do not improve. This does not isolate a causal speedup.

The experiment identifies an implementation avenue worth studying, but the split kernel remains opt-in. Profiling attributes similar aggregate time to gate, up and down projections and does not resolve instruction stalls. Better kernel scheduling and controlled, interleaved timing are concrete next steps; the present report uses only the completed measurements.

7 Discussion and conclusion

The main result is a working 27B-model deployment from a new packed representation. MORPH32-c removes 2.95 GB, or 18.31%, from the complete MLX 4-bit checkpoint while scoring 82.46% versus 84.04% on the fixed MMLU subset. MORPH32-3s offers a simpler quality–size point: 2.41 GB less storage and 83.60% MMLU. Both savings remain present in loaded model arrays. Achieving those results required fitting a constrained code across all 192 language MLP projections, packaging a standalone checkpoint and executing it directly in Metal without a complete dense MLP copy. The measured result is an end-to-end compression system, rather than a nominal tile bit rate alone.

The two layouts make the design tradeoff tangible. Reconstructing half of the third plane saves 16 bits per compact tile; storing that half independently improves the measured primary-model fidelity. The tested MLX 3-bit checkpoint is smaller and faster, while both MORPH profiles score higher on the evaluated MMLU, HumanEval and prediction tests. Different tensor scopes prevent an equal-storage or algorithm-only conclusion. The lossless metadata control further shows that some savings are exact, separate from the quality cost of MLP compression. Together, these controls explain which parts of the gain belong to the representation and which belong to packaging.

The current implementation still pays for bit extraction and reconstruction: generation and prefill are slower than the native MLX baselines. HumanEval falls relative to MLX 4-bit, stronger fitting has mixed effects, and the second-model study shows a larger perplexity loss. Evidence is limited to the evaluated text path, one primary model, one smaller model and one Apple M5 Pro. Improving the compact decoder and fitting across model sizes, then measuring with interleaved timing, are the clearest next steps. Even with these limits, MORPH-32 establishes that coupled sign planes can be fitted, stored and consumed as practical weights for a large language model, with a substantial measured storage reduction and an explicit account of its quality and speed costs.

8 References

- Lin et al. (2017). [Towards Accurate Binary Convolutional Neural Network](#).
- Frantar et al. (2023). [GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers](#).
- Lin et al. (2024). [AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration](#).
- Egiazarian et al. (2024). [Extreme Compression of Large Language Models via Additive Quantization](#).
- Tseng et al. (2024). [QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks](#).
- Xiao et al. (2023). [SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models](#).
- GGML contributors. [GGUF specification](#).
- MLX contributors. [Groupwise quantization and packed quantized matrix multiplication](#).
- lukaskremla. [Qwen3.8-27B-3bit-MLX. Pinned checkpoint](#).

A Reproducibility and scoring details

Inputs MLX 4-bit is `mlx-community/Qwen3.8-27B-4bit`, revision `3e6447f082e89cc7f0bc6e5441afd38dfce760ff`; BF16 targets are `Qwen/Qwen3.8-27B`, revision `1d4bf0f2ff6012fd82039f2fa52739d0dd7c60c0`. MLX 3-bit is `lukaskremla/Qwen3.8-27B-3bit-MLX`, revision `b3503b31b36125107e12a2b57fbe690c37bfc053`. Its architecture config differs only in quantization fields; tokenizer and chat-template files match byte-for-byte.

Calibration uses WikiText-2 validation tokens 32,768–32,895. Text scoring uses 4,096 WikiText-2 test tokens from offset 8,192 with fresh context. Code scoring uses a frozen CodeSearchNet Python test excerpt, 4,096 tokens from offset zero. Data selection, revisions and hashes are archived. Validation/test separation avoids direct calibration reuse; related Wikipedia domains and possible pretraining contamination remain limitations.

Scoring Perplexity is $\exp(\text{NLL})$. Mean KL uses full-vocabulary distributions with direction $D_{\text{KL}}(p_{4\text{-bit}} \parallel p_{\text{candidate}})$ on common teacher-forced contexts. Token agreement compares the most likely next tokens. Correlated token observations are not treated as independent confidence samples. No multi-step trajectory fidelity metric was computed.

MMLU uses five-shot dev examples and the single tokens for space-prefixed A/B/C/D. The prompt prefix is checked unchanged; no chat template or generated reasoning is used. HumanEval uses raw-prefix greedy completion, fixed top-level stop strings and a 512-token cap. Official tests run in a pinned, network-isolated, read-only container with timeouts. Its pass rate is one completion per task, conventionally `pass@1`. Wilson intervals describe finite-task counts, not training variance or equivalence.

HumanEval export correction The streaming detokenizer removed an initial space. Retained token IDs were decoded offline with the frozen tokenizer and unchanged stop strings; every corrected completion differs by exactly one leading space. All six configurations use the corrected exports. Original text, token histories and grading outputs remain linked by hashes. No inference or completion logic was changed by the correction.

Retrieval and timing Retrieval inserts ten fixed codes at 0, 25, 50, 75 and 100 percent of the background. Exact output-string matching scores the 50 prompts. A trial succeeds only when its code is recovered at all five positions. Timing measures ten synchronized runs per configuration; each has 4,096 input and 64 generated tokens at batch one. Cross-session ordering was not randomized. Resident bytes count loaded model arrays; peak MLX allocation includes cache and temporaries and must not be added to resident bytes.

Evidence The primary and smaller manifests contain 41 and 9 completed steps. `paper/data/research/current-20260911/checkpoint-index.json` maps plan checkpoints to raw outputs and audits. `paper/data/final-summary.json` supplies this manuscript’s tables and headline numbers. Regenerate the numerical summaries with:

```
uv run python paper/artifact.py
```

The command validates saved arithmetic, matched inputs, token-decoding chains and output hashes without inference. Format arithmetic and container details are specified above, and the source is rendered from the same audited data. Related-work citations link directly to the original publications.

B Complete numerical results

Table 6: **Memory and throughput for all primary configurations.** Peak is the maximum MLX allocator peak across the ten timing runs. Generation and prefill are median tokens/s. File size is in Table 5.

Configuration	Resident GB	Peak GB	Generation	Prefill
MLX 4-bit	15.13	18.26	15.92	417.71
MORPH32-c	12.19	15.33	13.35	348.88
MORPH32-3s	12.72	15.87	14.64	366.16
MLX 3-bit	11.77	14.90	22.26	492.09
3s, base fit	12.72	15.87	15.24	413.29
3s, metadata off	12.99	16.12	16.58	451.86

Table 7: **Changes relative to MLX 4-bit.** Δ NLL is nats/token; agreement is top-1 reference-token agreement, not task accuracy.

Configuration	Text Δ NLL	Code Δ NLL	Text agree.	Code agree.
MLX 4-bit	0.0000	0.0000	100.00%	100.00%
MORPH32-c	0.0480	0.0458	88.06%	92.16%
MORPH32-3s	0.0339	0.0269	89.82%	93.04%
MLX 3-bit	0.0924	0.0678	82.96%	88.33%
3s, base fit	0.0339	0.0277	89.72%	92.92%
3s, metadata off	0.0339	0.0269	89.82%	93.04%

Table 8: **Capability counts and descriptive Wilson intervals.** Interval endpoints are percentages. Retrieval is 10/10 trials for every configuration.

Configuration	MMLU count	95% interval	HE count	95% interval
MLX 4-bit	958/1140	81.80–86.05	126/164	69.80–82.63
MORPH32-c	940/1140	80.14–84.55	115/164	62.72–76.60
MORPH32-3s	953/1140	81.33–85.63	115/164	62.72–76.60
MLX 3-bit	896/1140	76.12–80.88	105/164	56.44–70.97
3s, base fit	953/1140	81.33–85.63	112/164	60.82–74.93
3s, metadata off	953/1140	81.33–85.63	115/164	62.72–76.60

Table 9: **Second-model check.** MORPH reduces storage but substantially worsens prediction quality on this model. This suite repeats the central measurements, not the capability benchmarks.

Qwen3-0.6B	File GB	Text PPL	Code PPL	Text KL	Gen. tok/s
MLX 4-bit	0.351	14.520	5.803	0.0000	219.65
MORPH32-3s	0.313	21.258	6.918	0.4129	204.92
3s, base fit	0.313	21.078	6.919	0.4071	204.45